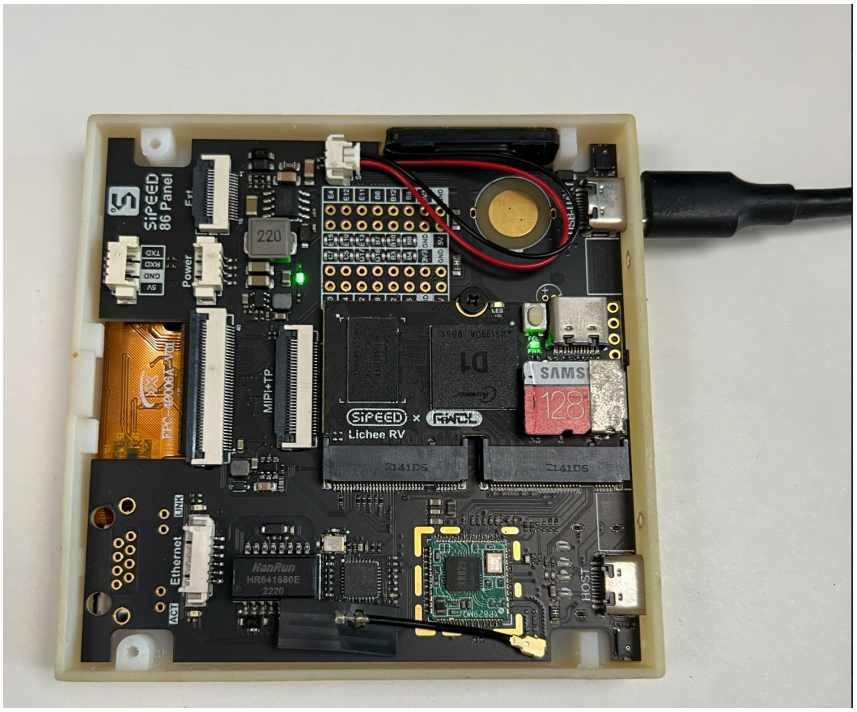
Allwinner D1 Operating-System Bring-Up and Benchmark Experiment

Linux vs RT-Thread RT-Smart vs Apache NuttX on the Sipeed Lichee RV / 86 Panel

Prepared by Lance Harvie | https://www.linkedin.com/in/lanceharvie/

All tests executed on the same Allwinner D1 / T-Head C906 hardware unless stated otherwise.



The physical Sipeed Lichee RV / 86 Panel used throughout the experiment.

Three operating systems Ubuntu Linux 24.04.1 LTS, RT-Smart 5.0.2, NuttX 13.0.1-RC0	**One hardware platform** Allwinner D1, T-Head C906 RV64, 512 MiB DDR3	**Final NuttX loaded result** p50 0.542 us, p99 0.583 us, max 0.708 us, 0 misses

This report documents the complete engineering path: baseline establishment, RT-Smart port stabilization, NuttX board-port creation, watchdog and timer diagnosis, benchmark development, measurement validation, and final cross-OS results.

Contents

1. Executive Summary

This experiment began with a simple question: how much of the behavior usually associated with a full embedded Linux stack is actually needed for a constrained embedded application, and what changes when the same hardware runs a userspace RTOS or a much lighter shared-address-space RTOS? The goal was not to produce a universal ranking. It was to put three materially different operating-system models on the same Allwinner D1 board, measure them with as much methodological consistency as practical, and document the engineering tradeoffs that appeared in real bring-up.

The three environments were Ubuntu Linux 24.04.1 LTS, RT-Thread RT-Smart 5.0.2, and Apache NuttX 13.0.1-RC0. Linux served as the mature protected-process baseline. RT-Smart represented an embedded operating system with MMU-backed userspace and process isolation. NuttX was intentionally brought up first in FLAT S-mode, where all tasks share one address space, because the priority was to establish a stable D1 hardware port before attempting MMU/user-mode work.

Area	Linux	RT-Smart 5.0.2	NuttX 13.0.1-RC0
Protection model	Protected kernel + user processes	MMU-enabled userspace + process isolation	FLAT S-mode, shared address space
clock_gettime net	121.28 ns	~10.7 us	51.606 ns
Contended mutex p50	21.666 us	6.833 us	1.042 us
Semaphore p50	21.208 us	13.250 us	1.167 us
pthread create p50	265.333 us	1063.958 us	10.000 us
1 ms loaded latency	64.09 us p50, 76.51 us p99, 228.72 us max (FIFO)	Not comparable in frozen 100 Hz build	0.542 us p50, 0.583 us p99, 0.708 us max

Central finding

The experiment did not produce a simple "RTOS is faster than Linux" result. Linux had exceptionally efficient userspace fast paths and the broadest feature set. RT-Smart delivered genuine process isolation in a much smaller embedded environment but exposed incomplete POSIX/runtime behavior in several areas. NuttX produced the lowest synchronization and periodic-wakeup latencies by a large margin, but it did so in a FLAT shared-address-space configuration with fewer Unix-like features and no process isolation.

- **Linux:** strongest feature coverage, mature process semantics, fast uncontended primitives, and respectable real-time behavior with SCHED_FIFO even on a non-PREEMPT_RT kernel.
- **RT-Smart:** a useful middle ground architecturally; process isolation worked and synchronization handoffs were faster than Linux, but pipe, PI mutex, fork/waitpid, and 1 ms periodic testing were limited in the frozen configuration.
- **NuttX:** extremely low task-creation, handoff, and periodic-wakeup latency after the D1 timer path was corrected; however, these results describe a FLAT shared-address-space build, not a protected userspace system.
- **Engineering lesson:** timer correctness and benchmark instrumentation mattered as much as the operating-system code. A one-count Timer1 programming error and UART diagnostics were both large enough to invalidate sub-microsecond latency conclusions until corrected.

2. Experiment Objective and Design

The experiment was designed around a single fixed board so that CPU core, memory subsystem, board layout, oscillator sources, serial path, and boot media remained constant. The operating-system software and configuration were changed; the physical platform was not.

The benchmark scope emphasized operating-system behavior that embedded engineers regularly encounter when deciding whether they need a full Linux environment, a userspace RTOS, or a minimal RTOS. The work therefore concentrated on clocks, synchronization, task creation, periodic scheduling, memory footprint, and a small set of POSIX process and IPC capabilities.

2.1 Design principles

- **Same physical hardware:** all three environments ran on the same Allwinner D1 / T-Head C906 platform.
- **Keep the OS honest:** unsupported features were recorded as unsupported or disabled rather than enabled simply to improve the comparison.
- **Separate observation from scheduling:** the RISC-V TIME counter (`rdtime`) at 24 MHz was used as the high-resolution observation source, while OS clock APIs controlled the requested scheduling deadlines.
- **Preserve architecture differences:** NuttX FLAT results were not presented as equivalent to RT-Smart or Linux process-isolation measurements.
- **Validate the measurement system:** early-return checks, scheduling readback, clock-domain alignment, and end-of-run drift were treated as part of the benchmark rather than afterthoughts.
- **Stop when the evidence was sufficient:** the study deliberately froze configurations once they were stable enough to measure, rather than continually tuning each OS for a higher score.

2.2 Benchmark families

Benchmark	Purpose	Typical sample count
highres	Measure aggregate rdtime read overhead and validate the 24 MHz observation counter	1,000,000
clock	Measure clock_gettime(CLOCK_MONOTONIC) call overhead	1,000,000
mutex	Uncontended pthread mutex lock/unlock fast path	10,000,000 pairs
mutex-contended	Release-to-acquire handoff between two threads	100,000 + warmup
semaphore	sem_post to sem_wait return handoff	100,000 + warmup
thread-create	pthread_create to child first instruction	10,000 + warmup
memory	OS-appropriate thread/process and payload footprint	multiple payload sizes
latency	1 ms absolute periodic wake-up lateness	20,000 + 100 warmup
latency-load	Same periodic test with sustained single-core CPU load	20,000 + 100 warmup

3. Hardware Platform and Boot Environment

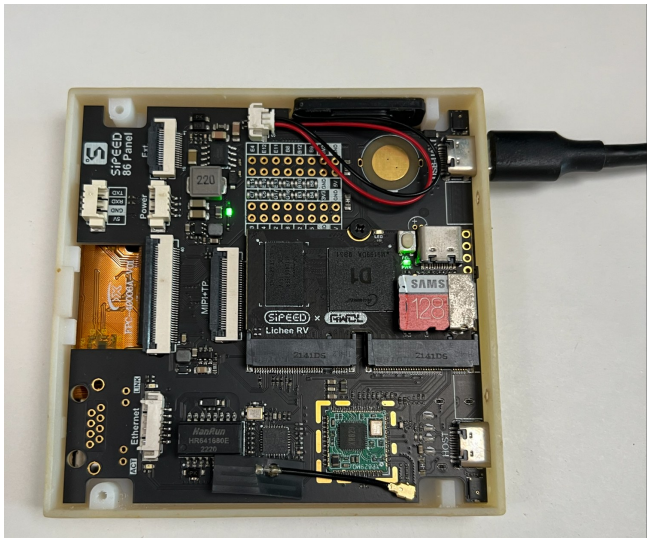


Figure 1. Physical board used for the experiment.

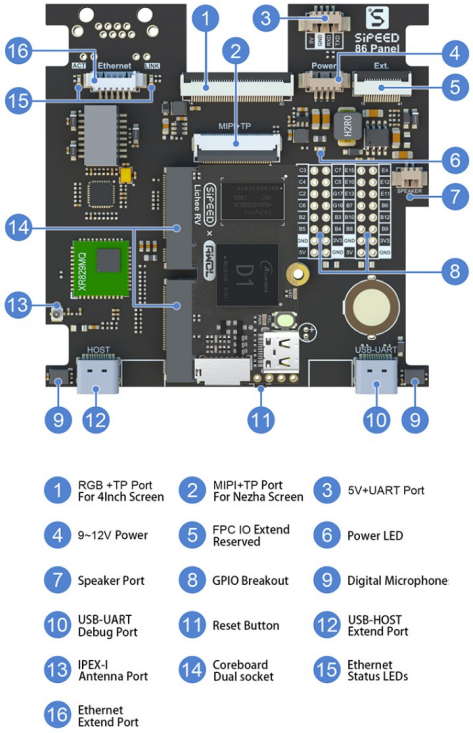


Figure 2. Sipeed 86 Panel carrier-board connector map.

3.1 Hardware summary

Component	Test configuration
SoC	Allwinner D1 / sun20i-d1
CPU	T-Head / XuanTie C906, RV64, single core, approximately 1.008 GHz
ISA used	rv64imafdc (NuttX adds zicsr + zifencei in compiler target)
Memory	512 MiB physical DDR3
Timebase	24 MHz RISC-V TIME counter
Serial console	UART0, 115200 baud, 8N1
Boot media	microSD; existing SPL/OpenSBI/U-Boot chain retained
NuttX link/load	0x40200000
RT-Smart kernel virtual/load region	0x45000000-based bring-up

3.2 Common boot chain

The development strategy deliberately preserved the board's working boot infrastructure. New RT-Smart and NuttX images were staged beside Ubuntu on the development SD card and loaded manually from U-Boot. This reduced the number of variables during bring-up and made recovery straightforward.

```
D1 BootROM -> U-Boot SPL -> OpenSBI -> U-Boot -> selected OS image

Typical NuttX boot:
ext4load mmc 0:1 0x48000000 /boot/nuttx-d1-benchmark-v4.img
bootm 0x48000000 - ${fdtcontroladdr}
```

4. Linux Baseline

Ubuntu Linux established the reference point for a mature protected-process operating system on the D1. The tested image was Ubuntu 24.04.1 LTS with Linux 6.8.0-41-generic. The kernel reported PREEMPT_DYNAMIC support with voluntary preemption available, but this was not a PREEMPT_RT kernel. The CPU frequency governor was held at the 1.008 GHz performance setting for the benchmark runs.

Item	Linux baseline
Distribution	Ubuntu 24.04.1 LTS
Kernel	6.8.0-41-generic, RISC-V, PREEMPT_DYNAMIC
Kernel tick	HZ=250
Compiler	GCC 13.3; -O2 -march=rv64gc -mabi=lp64d -static
Boot-time scale	~38.6 MB vmlinuz + ~18.7 MB initrd; 107 processes / 135 threads in the observed baseline

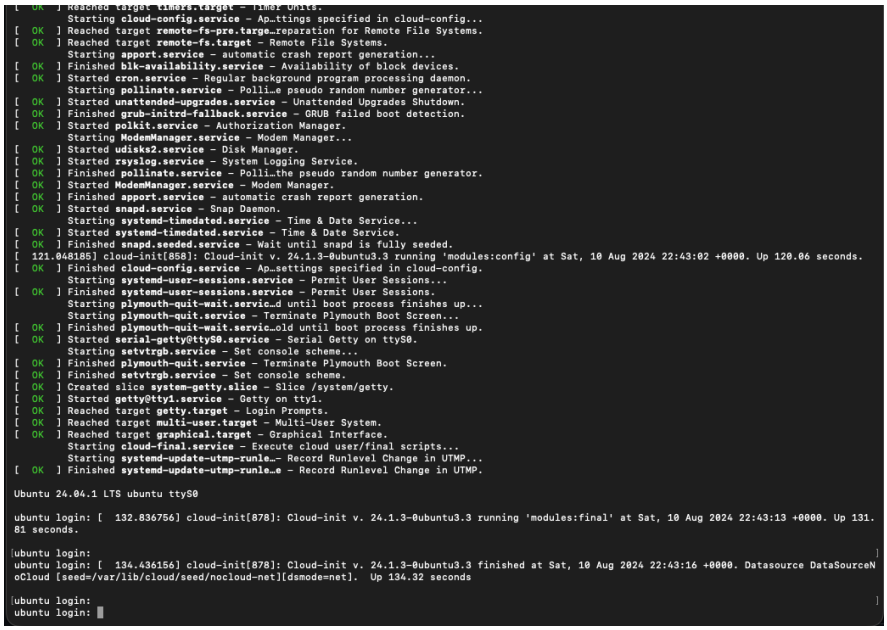


Figure 3. Ubuntu 24.04.1 LTS reaching the serial login console on the D1.

4.1 Linux performance baseline

Test	Result
clock_gettime net	121.28 ns/call
direct clock_gettime syscall net	971.23 ns/call
uncontended pthread mutex	86.46 ns per lock/unlock pair
contended mutex normal	p50 21.666 us, p99 28.208 us, max 223.916 us
PI mutex	p50 41.042 us, p99 61.375 us, max 270.250 us
semaphore handoff	p50 21.208 us, p99 28.541 us, p99.9 81.084 us, max 230.833 us
pipe handoff	p50 37.042 us, p99 62.750 us, max 264.750 us
pthread create	p50 265.333 us, p99 863.250 us
fork	p50 1501.667 us, p99 2047.041 us
posix_spawn, static child	~4.10 ms median on ext4 and tmpfs

4.2 One-millisecond periodic latency

Linux mode	p50	p99	p99.9	max	missed
CFS idle	129.43 us	151.54 us	533.88 us	1668.26 us	5
SCHED_FIFO 80 idle	67.46 us	89.05 us	128.46 us	233.75 us	0
CFS loaded	116.42 us	3741.38 us	6043.88 us	10755.59 us	1284 / 6.42%
SCHED_FIFO 80 loaded	64.09 us	76.51 us	94.22 us	228.72 us	0

Linux interpretation

Linux demonstrated two distinct characteristics. Its uncontended userspace primitives were extremely efficient, while real-time wake-up tails depended strongly on scheduling class. Under sustained load, ordinary CFS scheduling produced millisecond-scale tail latency and misses; SCHED_FIFO eliminated misses and held the p99 below 77 us on the same kernel.

5. RT-Thread RT-Smart Bring-Up

The RT-Smart leg required the most traditional BSP bring-up work. The target was RT-Thread v5.0.2 with RT-Smart userspace on the D1. The result was a genuine U-mode userspace environment with process isolation, a POSIX-like runtime, ROMFS/devfs, and a working shell. The D1 fixes were collected into PR RT-Thread/rt-thread#11714 from branch fix/allwinner-d1-rtsmart.

5.1 Key bring-up changes

- Migrated the D1 MMU setup to the current `rt_kernel_space` / `rt_hw_mmu_map_init` / `rt_hw_mmu_setup` model.
- Corrected kernel virtual-address placement and identity mapping so the kernel heap and page pool were covered.
- Moved heap initialization ahead of interrupt initialization where required by the current code path.
- Enabled Serial V2, corrected the console device name, removed a stale UART init call, and standardized the D1 console at 115200 baud.
- Adapted D1 SDMMC/partition handling from the D1s port and disabled incompatible FAL assumptions.
- Resolved duplicate reset and BSS-init paths and added linker symbols required by the contemporary RT-Smart runtime.
- Separated the OpenSBI timer/FDT issue from the RT-Thread PR so the OS change set remained focused on the D1 BSP itself.

5.2 RT-Smart boot and memory baseline

Metric	Observed RT-Smart baseline
Kernel heap total	52,428,600 bytes
Kernel heap used after stabilization	~240 KB
Page pool installed / free	51,196 KiB / 48,420 KiB
Kernel threads	8 baseline kernel threads
Clock tick	100 Hz / 10 ms
CLOCK_MONOTONIC resolution	10 ms
Simple process fixed overhead	~120 KiB measured rough fixed runtime cost
Memory reclamation	Payload/page allocations returned to baseline in tested process-memory runs

5.3 Process-isolation result

Unlike the later NuttX FLAT build, RT-Smart executed real userspace ELFs in U-mode. Test applications showed separate process allocation, user stacks, and heap payloads, with resources returning to the baseline after exit. For larger allocations the measured process cost was approximately 124 KiB of fixed overhead plus the touched payload. That architecture distinction is central when interpreting the later NuttX speed results.

6. RT-Smart Benchmark Results and Limitations

Test	RT-Smart 5.0.2 retained result
rdtime aggregate overhead	~1.989 ns/read net (aggregate throughput; TIME ticks are not CPU cycles)
clock_gettime net	~10.7 us/call
uncontended mutex	~7.20 us/pair
contended normal mutex	p50 6.833 us, p99 7.708 us, p99.9 7.875 us, max 17.167 us
semaphore handoff	p50 13.250 us, p99 14.458 us, p99.9 15.458 us, max 27.167 us
pthread create	p50 1063.958 us, p99 1113.875 us, max 1133.792 us
PI mutex	Unsupported in the v5.0.2 futex backend
pipe	Runtime interface non-operational in the tested userspace stack
fork	Child creation worked, but waitpid result/cleanup behavior was abnormal under the benchmark path
1 ms periodic test	Invalid in frozen 100 Hz configuration: all 20,000 requests returned before the requested 1 ms deadline

6.1 What the failures revealed

- **Pipe:** the userspace POSIX pipe interface returned descriptor state that could not support a reliable handoff benchmark. The failure was recorded rather than patched solely to obtain a score.
- **Fork/waitpid:** the original benchmark used a pipe to return the child timestamp, so early EIO results were benchmark-path failures rather than proof that `fork()` itself failed. Later smoke testing showed child creation and reclamation but abnormal `waitpid()` return behavior.
- **PID-slot exhaustion:** a tight fork loop outran deferred LWP reclamation and filled the 30-slot process table, eventually exposing a kernel robustness fault. The destructive loop was not repeated.
- **Priority inheritance:** header/API presence did not imply a working PI futex backend. The benchmark reported the feature as unsupported.
- **Periodic latency:** a 1 ms deadline cannot be fairly measured through a 10 ms POSIX clock/sleep granularity. Zero-latency-looking early-return results were explicitly rejected as invalid.

RT-Smart takeaway

RT-Smart occupied the architectural middle ground of the experiment. It preserved process isolation and a userspace programming model while remaining far smaller than Linux. Its blocking synchronization latency was better than Linux in the tested runs, but fast-path overhead and incomplete POSIX/runtime behavior limited the final benchmark matrix.

7. Creating the NuttX D1 Port

At the tested Apache NuttX revision, no upstream Allwinner D1 / Lichee RV / 86 Panel board target was present. The experiment therefore created a local board port on branch `local/allwinner-d1-bringup`. Ox64 and QEMU were used only as architectural references; neither was substituted for the D1 hardware.

Component	Value
NuttX base commit	b7c8430de698e43386683e12f10680c48e90ba26
nuttx-apps commit	cc3c3aa3169b9d2a137f271ccbb84e4ab590ef75
Local D1 branch commit	aa353a1b2a55cb0d6b8a5f51a7055373017cc907
Board target	lichee-rv-86-panel:nsh
Compiler	riscv-none-elf-gcc 14.3.0
Architecture flags	-march=rv64imafdc_zicsr_zifencei -mabi=lp64d -mcmodel=medany
Initial build mode	CONFIG_BUILD_FLAT=y; CONFIG_ARCH_USE_S_MODE=y

7.1 Minimal hardware scope

- UART0 console on PB8/PB9 at 115200 8N1.
- S-mode execution under the existing OpenSBI firmware.
- PLIC interrupt routing for the single C906 hart.
- A 126 MiB initial RAM window from 0x40200000 to 0x48000000, with NuttX linked and entered at 0x40200000.
- NSH shell, pthreads, mutexes, semaphores, clocks, heap, and procfs sufficient for bring-up and benchmarking.
- No display, touch, Ethernet, Wi-Fi, audio, USB, or SD filesystem work was required for the benchmark milestone.

7.2 Why FLAT mode first

NuttX supports richer protected/kernel build models on RISC-V, and upstream C906-class ports demonstrate Sv39/MMU mechanisms. However, the D1 port did not yet have the addrenv, user linker, cache/MMU mapping, and ELF-user-space work needed for a protected benchmark. Starting in FLAT mode minimized bring-up variables and made the timer, interrupt, scheduler, and benchmark behavior easier to isolate. The consequence is important: NuttX performance numbers in this report describe a shared S-mode address space, not a Linux-like process model.

8. NuttX Bring-Up: Watchdog and Timer Debugging

The first local NuttX image reached `nsh>`, which initially looked like success. The board then rebooted within roughly 20 seconds. Once that reset was separated from the shell and sleep behavior, the root cause became clear: two independent platform issues were present.

8.1 Inherited 16-second watchdog

The working RT-Smart D1 BSP contained an unusually strong clue: it explicitly documented that U-Boot starts the D1 RISC-V watchdog with a 16-second timeout and disables it during board initialization. NuttX did not touch that watchdog. A U-Boot register read showed `0x06011018 = 0x000000b1`, where bit 0 enabled the watchdog and timeout selector 0xb corresponded to 16 seconds.

```
U-Boot transient test:
md.l 06011018 1        # observed 000000b1
mw.l 06011018 16aa0000 1
md.l 06011018 1        # enable bit cleared
```

With the watchdog disabled, NuttX remained alive beyond 60 seconds, confirming the reset source. The permanent port fix writes `0x16aa0000` to the watchdog mode register very early in C startup, followed by a full I/O fence.

8.2 SBI TIME accepted, but STIP never arrived

After the watchdog issue was removed, `sleep 1` still failed to wake. The free-running time source advanced, but timed waits did not expire. A timer diagnostic build showed that NuttX attached the supervisor-timer IRQ, enabled STIE, programmed a 24,000-count absolute deadline through SBI TIME, and still never received the first supervisor timer interrupt.

Diagnostic observation	Hardware result
SBI spec	1.0
OpenSBI implementation	ID 1, version 0x00010003 (OpenSBI 1.3)
SBI TIME probe	value=1
SBI set_timer return	error=0, value=0
First deadline delta	24,000 TIME counts = 1 ms at 24 MHz
TIME after long diagnostic print	deadline already exceeded by ~22.97 ms
sip.STIP after deadline	0
FIRST STIMER IRQ marker	never observed

Conclusion from the SBI diagnostic

The OS-side timer setup was internally consistent and OpenSBI accepted the request, but supervisor timer pending was not injected on the running platform/firmware path. Rather than continue modifying firmware, the experiment moved NuttX to the native Allwinner D1 Timer1 peripheral, which was already a proven strategy in the working RT-Smart environment.

9. NuttX Native D1 Timer and the Off-by-One Discovery

The final NuttX scheduler tick used D1 Timer1 directly. The timer runs from OSC24M through the PLIC and calls the standard NuttX scheduler timer processing from the ISR. This removed dependence on the non-working STIP injection path.

Timer item	Final NuttX value
Peripheral base	0x02050000
Channel	Timer 1
Control register	0x02050020
Interval register	0x02050024
IRQ enable/status	0x02050000 bit 1 / 0x02050004 bit 1 W1C
Clock	OSC24M, prescaler /1
PLIC source	76
NuttX IRQ	101 (RISCV_IRQ_SEXT 25 + 76)
Configured tick	1000 Hz / 1000 us
Final interval value	23,999

9.1 Why 24,000 was wrong

The first native-timer build programmed an interval value of 24,000 for a nominal 1 ms period. A phase-aligned 20.122-second latency run then showed 20,126 excess `rdtime` ticks. The number of scheduler intervals during the same period was about 20,122. The mismatch was therefore almost exactly one 24 MHz input clock per OS tick.

Measured v3 drift	**Prediction if period = 24,001 clocks**	**After interval = 23,999**
+20,126 TIME ticks +838.583 us over ~20.122 s	20,122 intervals x 1 extra clock = 20,122 ticks	v4 drift = -3 TIME ticks -0.125 us

The D1 timer counts down to a terminal zero before periodic reload. On the physical board, writing 24,000 therefore produced an effective 24,001-clock period. Changing the interval register to 23,999 produced exactly 24,000 input clocks per scheduler tick and removed the accumulated drift. This was not a cosmetic calibration: without the correction, the 1 ms latency benchmark mixed a drifting OS clock with a stable 24 MHz observation timer and could not support sub-microsecond conclusions.

Critical engineering insight

A one-count hardware-timer interpretation error produced only about 41.7 ppm frequency error, yet over a 20-second benchmark it accumulated to nearly 0.84 ms. The measurement framework was sensitive enough to expose it, and the result became trustworthy only after the timer itself was corrected.

10. Benchmark Methodology and Measurement Validation

The benchmark suite evolved during the experiment. The final objective was not merely to print numbers, but to ensure that each number had a defensible relationship to an operating-system operation. This was especially important for periodic wake-up latency, where the OS clock had millisecond resolution while `rdtime` provided a 41.667 ns observation quantum.

10.1 High-resolution observation source

The D1 RISC-V TIME counter runs at 24,000,000 Hz. NuttX validated this at runtime by sleeping for approximately one second and measuring 24,009,164 TIME ticks, a ratio of 1.000382 relative to the nominal value. TIME ticks were always described as TIME-counter ticks, not CPU cycles.

10.2 Periodic latency alignment

A naive mapping between a 1 ms quantized `CLOCK_MONOTONIC` reading and an immediately adjacent `rdtime` reading created apparent early wakeups hundreds of microseconds before the requested deadline. The corrected method polled for exact CLOCK_MONOTONIC transitions, bracketed each transition with two `rdtime` reads, selected the narrowest of eight transition brackets, and used the midpoint as the phase reference.

- Start alignment was captured before warmup and measurement, with no console output after the reference.
- The first deadline was placed 20 ms after the aligned reference; subsequent deadlines advanced by exactly 1,000,000 ns and 24,000 TIME ticks.
- The timestamp was captured immediately after `clock_nanosleep(..., TIMER_ABSTIME, ...)` returned.
- A separate CLOCK_MONOTONIC read checked POSIX deadline semantics. Semantic early returns had to remain zero.
- An end alignment measured clock-domain drift. The run was rejected if alignment uncertainty or drift exceeded predefined limits.
- No percentile distribution was published from a run that failed the validity checks.

10.3 Console perturbation

The v2 latency benchmark printed the start-alignment diagnostic before the end reference. At 115200 baud that output consumed enough time to delay/coalesce scheduler ticks and produced about 15.8 ms of apparent drift. Version 3 made the entire measurement interval silent. This removed most of the error and exposed the remaining one-count hardware-timer defect described in Section 9.

Validity condition	Final NuttX criterion
Alignment bracket	<= 240 TIME ticks (10 us)
Midpoint uncertainty	<= +/-120 ticks (5 us)
Semantic early returns	0
Warmup semantic early returns	0
Drift	Within start uncertainty + end uncertainty + 240 ticks
Sample count	20,000 measured + 100 warmup

11. NuttX Benchmark Results

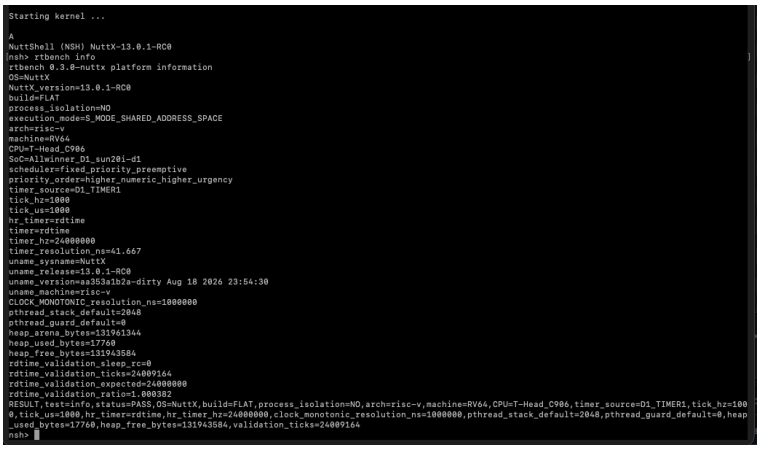


Figure 4. NuttX benchmark platform information from the serial console.

11.1 Core microbenchmarks

Metric	NuttX 13.0.1-RC0 FLAT
Build / scheduler	FLAT S-mode; fixed-priority preemptive
CLOCK_MONOTONIC resolution	1,000,000 ns (1 ms)
rdtime aggregate overhead	7.939 ns/read net
clock_gettime net	51.606 ns/call
uncontended mutex	154.823 ns/pair
contended mutex	p50 1.042 us, p99 1.333 us, p99.9 1.583 us, max 2.083 us
semaphore handoff	p50 1.167 us, p99 1.208 us, p99.9 1.542 us, max 2.042 us
pthread create	p50 10.000 us, p99 10.667 us, p99.9 11.000 us, max 11.250 us
PI mutex	Disabled in frozen config (CONFIG_PRIORITY_INHERITANCE=n)
Pipe	Disabled in frozen config
fork	Unsupported in this RV64 FLAT configuration
Process isolation	No

11.2 Memory accounting

Measurement	Observed NuttX FLAT delta
Baseline heap used	17,760 bytes
Live pthread	+2,400 bytes; returned completely after join
64 KiB payload	65,544 byte allocation delta; fully reclaimed
256 KiB payload	262,152 byte allocation delta; fully reclaimed
1 MiB payload	1,048,584 byte allocation delta; fully reclaimed

These values must not be placed in the same row as RT-Smart or Linux isolated-process overhead. The NuttX measurement describes a pthread and heap activity inside a shared address space, not the cost of constructing a protected process.

12. NuttX Periodic Latency Results

Mode	p50	p99	p99.9	max	missed	alignment drift
Unloaded	0.375 us	0.375 us	0.375 us	1.042 us	0 / 20,000	-3 ticks (-0.125 us
Loaded, FIFO120 vs lower-priority FIFO90 worker	0.542 us	0.583 us	0.708 us	0.708 us	0 / 20,000	+4 ticks (+0.167 u

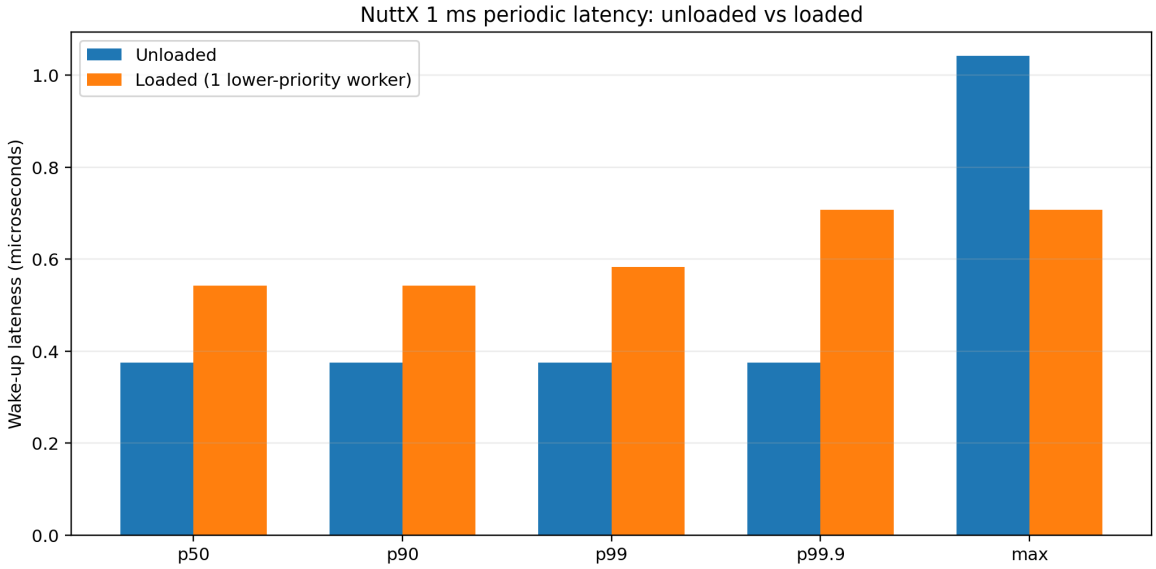


Figure 5. NuttX measured 1 ms wake-up lateness, unloaded and with one lower-priority continuous compute worker.

12.1 Interpreting sub-microsecond values

The reported central values are close to the phase-alignment uncertainty. The final alignment brackets were 17-18 TIME ticks, or roughly 0.71-0.75 us wide, with midpoint uncertainty of +/-0.375 us. The safe publication claim is therefore that the NuttX wake-up behavior was **sub-microsecond at the measurement floor with extremely tight tails**, rather than that the physical scheduler latency is known to exactly 0.375 us. The zero semantic early returns, zero missed periods, near-zero clock-domain drift, and maximum observed values around 1 us are the stronger evidence.

12.2 Loaded test scheduling

The loaded test explicitly requested and read back SCHED_FIFO priority 120 for the latency task and priority 90 for the continuous compute worker. Both were verified. Because NuttX uses higher numeric priority for higher urgency, the periodic task remained above the load worker on the single C906 core. This result therefore measures deterministic preemption of a lower-priority CPU-saturating workload, not fair-share contention between equal-priority tasks.

Figure 6. Unloaded v4 latency PASS output.

Figure 7. Loaded latency PASS output.

13. Cross-OS Performance Comparison

The comparison is most informative when the operating-system architecture is kept visible. The fastest number in a row is not automatically the best system for an application. Linux pays for rich kernel services, protection, VM, filesystem, and process semantics. RT-Smart pays for an embedded userspace/process model. The tested NuttX build avoids most of those boundaries.

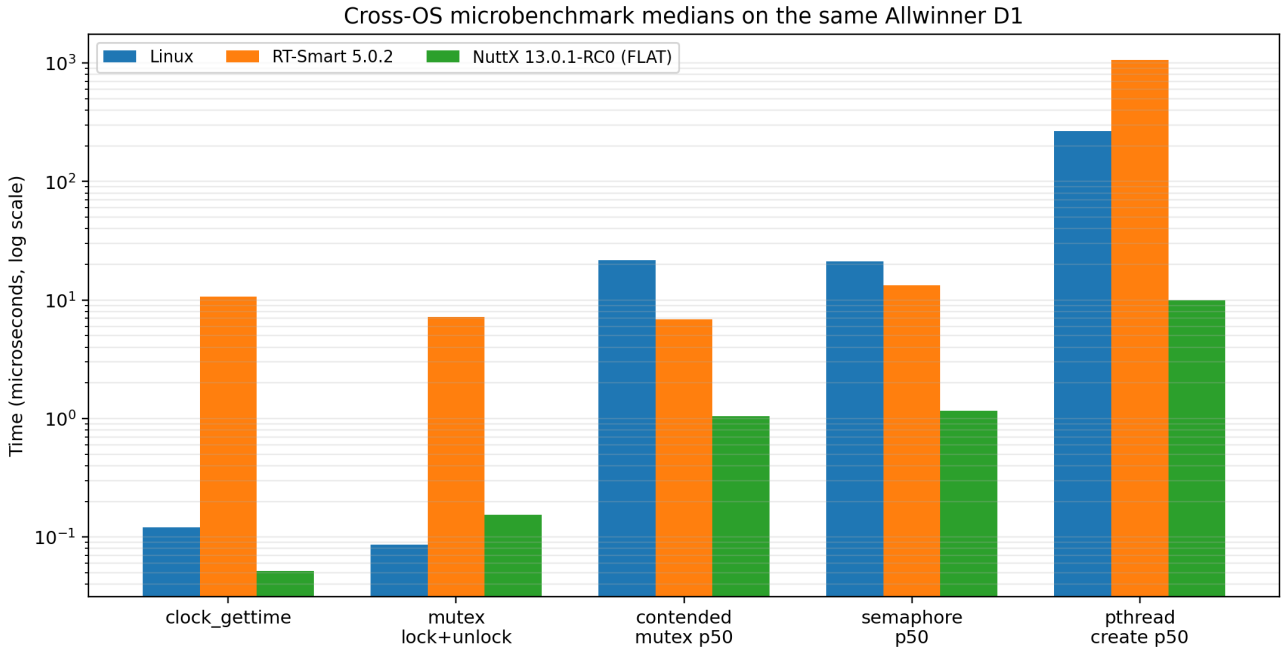


Figure 8. Median/fast-path results on a logarithmic scale. The NuttX numbers are from a FLAT shared-address-space build.

Metric	Linux	RT-Smart 5.0.2	NuttX 13.0.1-RC0
Protection model	Protected processes	MMU-backed userspace / process isolation	FLAT shared S-mode address space
clock_gettime	121.28 ns	~10.7 us	51.606 ns
uncontended mutex	86.46 ns	~7.20 us	154.823 ns
contended mutex p50 / p99	21.666 / 28.208 us	6.833 / 7.708 us	1.042 / 1.333 us
semaphore p50 / p99	21.208 / 28.541 us	13.250 / 14.458 us	1.167 / 1.208 us
pthread create p50 / p99	265.333 / 863.250 us	1063.958 / 1113.875 us	10.000 / 10.667 us
PI mutex	Supported	Unsupported in tested backend	Disabled in frozen config
Pipe	Working	Runtime interface broken in test	Disabled
fork/process creation	Working	Partial/abnormal waitpid path	Unsupported in FLAT
1 ms periodic test	Valid	Not valid at frozen 100 Hz tick	Valid after native Timer1 fix

14. Real-Time Behavior Under Load

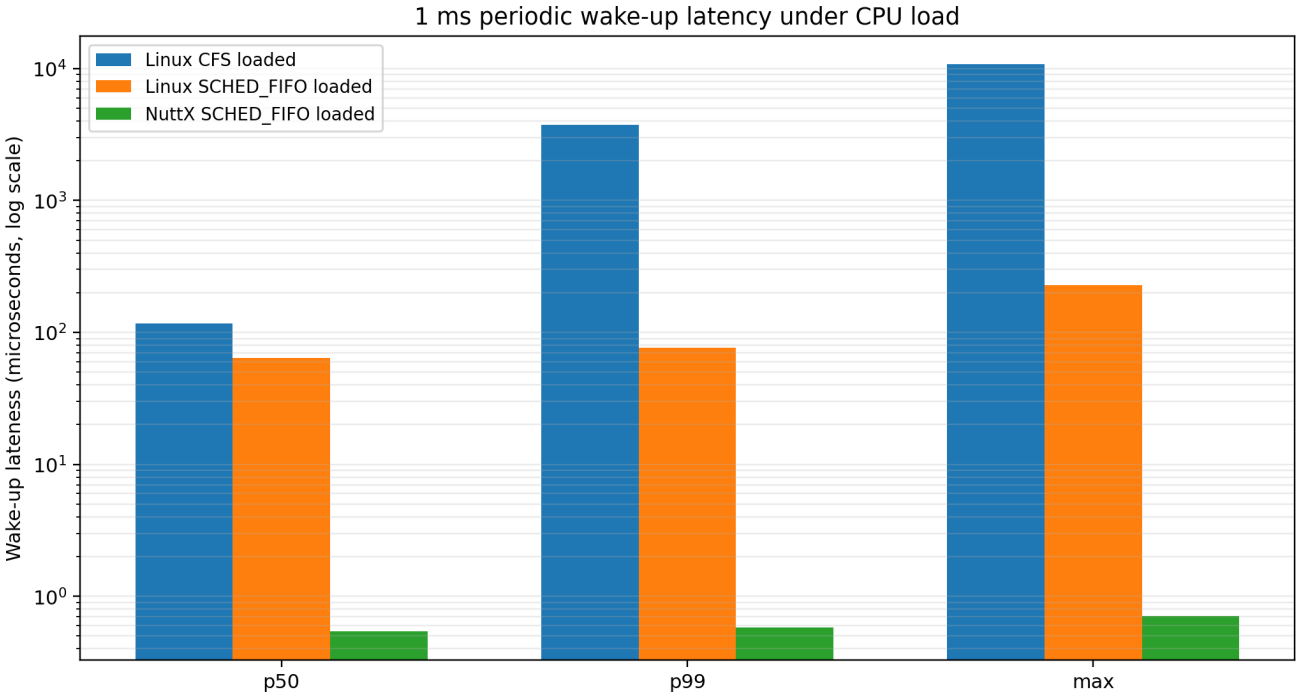


Figure 9. Loaded 1 ms wake-up lateness. RT-Smart is omitted because the frozen 100 Hz build could not represent the 1 ms deadline.

The Linux and NuttX loaded results demonstrate different meanings of "real time" on the same CPU. Ordinary Linux CFS scheduling retained a low median but developed millisecond-scale tails under sustained load and missed 6.42% of periods. Linux SCHED_FIFO eliminated those misses and held p99 to 76.51 us. NuttX, with a verified higher-priority periodic task over a lower-priority compute worker, remained at the sub-microsecond measurement floor with zero misses.

Important fairness caveat

This is not a claim that a FLAT NuttX application is universally 100x "faster" than Linux. The NuttX path is materially simpler: no user/kernel address-space transition, no protected process boundary, a fixed-priority scheduler, and a tiny active runtime. Linux is simultaneously delivering a far richer execution environment. The latency result quantifies that architectural tradeoff on this board.

14.1 Directional ratios

Comparison	Approximate ratio
Linux FIFO loaded p50 / NuttX loaded p50	64.09 / 0.542 = 118x
Linux FIFO loaded p99 / NuttX loaded p99	76.51 / 0.583 = 131x
Linux FIFO loaded max / NuttX loaded max	228.72 / 0.708 = 323x
RT-Smart contended mutex p50 / NuttX p50	6.833 / 1.042 = 6.6x
Linux contended mutex p50 / NuttX p50	21.666 / 1.042 = 20.8x
Linux pthread-create p50 / NuttX p50	265.333 / 10.000 = 26.5x

These ratios are descriptive of the tested configurations only. Scheduler policy, process model, system-call boundary, and feature set differ between systems.

15. Capability and Architecture Matrix

Capability	Linux	RT-Smart 5.0.2	NuttX FLAT
Protected user processes	Yes	Yes	No
MMU/process isolation	Yes	Yes	No in tested build
ELF userspace loading	Yes	Yes	Disabled in tested build
pthread mutex/semaphore	Yes	Yes	Yes
Priority-inheritance mutex	Yes	No in tested futex backend	Disabled by config
SCHED_FIFO / fixed priority	Yes	Requested, but readback unreliable in benchmark	Yes; request and readback verified
Pipe	Yes	Symbol/syscall present but runtime benchmark non-operational	Disabled by config
fork	Yes	Child creation observed; waitpid semantics abnormal	Unsupported in RV64 FLAT
posix_spawn	Yes	Not safely benchmarked	Compiled for built-ins, not Linux-equivalent process creation
1 ms POSIX absolute sleep	Yes	Not representable with 10 ms clock resolution	Yes
Native D1 periodic timer in final test	Kernel platform timer infrastructure	Yes in working RT-Smart build	Yes, D1 Timer1 via PLIC

15.1 The protection-model effect

The most consequential difference is not a single API; it is the protection model. Linux and RT-Smart impose boundaries between the application and privileged kernel state. The tested NuttX configuration does not. That makes NuttX thread creation and synchronization especially light, but it also means a memory error in one task can corrupt another task or the kernel-visible shared address space. For applications where fault containment, dynamic process loading, complex filesystems, networking stacks, or broad third-party software compatibility dominate, Linux retains an advantage that is not represented by a latency chart.

16. Interpretation: What the Experiment Actually Says

The experiment produced a more nuanced result than the initial premise. There was no monotonic progression where "smaller OS" simply meant "faster at everything." Each system was optimized for a different cost structure.

Linux

Strength: mature process model, strongest POSIX and I/O coverage, very efficient uncontended pthread/clock fast paths, and usable deterministic behavior with SCHED_FIFO.
Cost: higher blocking/wakeup latency and a much larger runtime environment.

RT-Smart

Strength: process isolation and userspace in a compact embedded system; handoff latency materially better than Linux in this experiment.
Cost: expensive POSIX fast paths and incomplete/fragile behavior in PI, pipe, fork/waitpid, and fine-grained timer tests.

NuttX FLAT

Strength: exceptionally low and tight synchronization, thread-creation, and periodic-wakeup latency after the D1 timer path was corrected.
Cost: no process isolation, disabled/unsupported Unix-like features, and a benchmark result that describes a much simpler execution model.

16.1 Why Linux won some fast paths

Linux `pthread_mutex_lock/unlock` on an uncontended normal mutex is heavily optimized and can complete entirely in userspace. That is why the Linux 86.46 ns pair beat the NuttX 154.823 ns pair even though NuttX was dramatically faster once blocking and scheduler handoff entered the picture. Likewise, a mature vDSO-style clock path can make Linux `clock_gettime` inexpensive without implying low scheduler wake-up latency.

16.2 Why NuttX dominated blocking handoffs

In the tested FLAT build, synchronization and scheduling operate inside a small fixed-priority environment without process isolation or a conventional protected system-call boundary. Once a higher-priority task becomes runnable, the path from interrupt or primitive release to execution is correspondingly short. The loaded latency run was designed to make that property visible: the higher-priority periodic task consistently preempted the lower-priority compute worker.

16.3 Why RT-Smart is still interesting

RT-Smart did not win the numerical table, but its architecture is arguably the most relevant comparison for teams seeking Linux-like separation without Linux-scale complexity. Its process isolation worked, memory was reclaimed correctly, and blocking synchronization beat Linux. The gaps uncovered in the v5.0.2 D1 userspace stack are therefore implementation and maturity findings, not evidence that the architectural model is unsound.

17. Limitations and Threats to Validity

- **NuttX build model:** the tested build is FLAT. Results cannot be assumed to hold for a future NuttX KERNEL/Sv39 userspace configuration.
- **Linux kernel:** the baseline used PREEMPT_DYNAMIC, not PREEMPT_RT. A PREEMPT_RT kernel is an obvious follow-up and may materially change Linux tail latency.
- **RT-Smart 1 ms test:** the frozen 100 Hz clock resolution made the requested 1 ms POSIX absolute sleep invalid. No RT-Smart 1 ms latency score is claimed.
- **Scheduling equivalence:** handoff tests preserved the same general algorithms, but scheduler priority semantics and exact controller/worker relationships differ across operating systems. Cross-OS ratios should be interpreted directionally.
- **Clock measurement floor:** NuttX sub-microsecond latency values are close to the +/-0.375 us phase-reference uncertainty. The robust claim is extremely tight sub-microsecond behavior with ~1 us observed maxima and zero missed periods.
- **Aggregate counter-read overhead:** reported `rdtime` overhead can be fractional in TIME ticks because it is an average over a large loop. It is not the latency of a single hardware counter tick and is not a CPU-cycle measurement.
- **Feature coverage:** disabled or broken APIs were not enabled/patched solely to make the matrix complete. This makes the feature comparison honest but incomplete.
- **Single board:** the study is a controlled case study on one D1/C906 platform. Different RISC-V cores, cache architectures, interrupt controllers, memory speeds, and compiler versions may produce different ratios.

Publication note

The report intentionally retains failed and invalid intermediate results where they illuminate methodology. In particular, the watchdog reset, SBI TIME failure, console-induced drift, and timer off-by-one are part of the engineering evidence, not noise to be removed from the narrative.

18. Engineering Lessons Learned

A shell prompt is not a completed bring-up

NuttX reached NSH early, yet an inherited watchdog still guaranteed a reset and the scheduler timer could not wake sleeping tasks. A useful acceptance test must exercise timed blocking, interrupts, and sustained uptime.

Bootloader state is part of the OS environment

The D1 watchdog state survived into the payload. RT-Smart already contained the workaround; NuttX initially did not. OS bring-up must account for inherited peripheral state, not only reset defaults.

"SBI call succeeded" does not prove timer delivery

The TIME extension was advertised and set_timer returned success, but STIP remained clear after the deadline. The diagnostic had to observe the pending interrupt condition, not just the API return value.

Measurement code can perturb real-time behavior

A long UART line at 115200 baud was sufficient to create millisecond-scale clock-domain drift during a benchmark. Silent measurement intervals are essential when the expected latency is microseconds or less.

Tiny timer errors accumulate

A one-count interval interpretation error at 24 MHz was only ~41.7 ppm but accumulated to ~0.84 ms over a 20-second run. Long-run phase checks can reveal defects that short scope-style tests miss.

Feature symbols are not runtime guarantees

RT-Smart pipe/fork and NuttX PI/pipe examples showed why capability matrices must distinguish compiled symbols, configuration, and proven runtime behavior.

Architecture determines what a benchmark number means

NuttX thread creation at 10 us is impressive, but it is also a shared-address-space task creation path. Comparing it to Linux fork or RT-Smart process creation without context would be misleading.

19. Recommended Next Steps

The current experiment is complete enough to support a technical article or conference-style engineering report. The following extensions would answer the remaining architectural questions rather than simply produce more microbenchmark numbers.

19.1 NuttX protected userspace

- Implement the D1 Sv39/MMU/addrenv path using the SG2000/JH7110/QEMU kernel-build references already identified.
- Enable ELF userspace loading and measure process/task overhead, syscall cost, synchronization, and periodic latency again.
- Repeat the exact benchmark suite to quantify how much of the current NuttX advantage comes from FLAT mode versus scheduler/runtime design.

19.2 RT-Smart completeness

- Resolve the userspace pipe/fcntl behavior and re-run pipe handoff.
- Investigate waitpid return semantics and deferred LWP cleanup without aggressive exhaustion loops.
- Evaluate a configuration with a 1 kHz tick or another supported high-resolution sleep mechanism, then repeat the phase-aligned latency methodology.
- Assess whether PI mutex support can be added to the futex backend without changing the benchmark semantics.

19.3 Linux real-time follow-up

- Build a comparable PREEMPT_RT kernel for the D1 and repeat idle/loaded 1 ms latency tests.
- Retain SCHED_OTHER and SCHED_FIFO baselines so the cost/benefit of PREEMPT_RT is visible.
- Optionally test CPU affinity, IRQ affinity where applicable, and reduced background userspace to distinguish kernel from distribution effects.

19.4 Preserve the benchmark as a reusable board harness

The benchmark is now more valuable than any single result. It contains capability reporting, high-resolution observation, scheduling verification, percentile statistics, safe task-creation pacing, phase alignment, semantic early-return detection, and drift validation. Reusing this harness on other RISC-V boards, Cortex-A embedded Linux systems, and microcontroller-class RTOS targets would build a more useful cross-platform dataset.

Appendix A. Detailed Result Matrix

Metric	Linux	RT-Smart	NuttX
High-res timer	TIME @ 24 MHz	TIME @ 24 MHz	TIME @ 24 MHz
clock_gettime resolution	High-resolution Linux clock path	10 ms	1 ms
clock_gettime net	121.28 ns	~10.7 us	51.606 ns
Uncontended mutex	86.46 ns/pair	~7.20 us/pair	154.823 ns/pair
Contended mutex p50	21.666 us	6.833 us	1.042 us
Contended mutex p99	28.208 us	7.708 us	1.333 us
Contended mutex max	223.916 us	17.167 us	2.083 us
Semaphore p50	21.208 us	13.250 us	1.167 us
Semaphore p99	28.541 us	14.458 us	1.208 us
Semaphore max	230.833 us	27.167 us	2.042 us
pthread create p50	265.333 us	1063.958 us	10.000 us
pthread create p99	863.250 us	1113.875 us	10.667 us
Pipe handoff	p50 37.042 us, p99 62.750 us	Runtime benchmark failed	Disabled
PI mutex	Supported; p50 41.042 us	Unsupported	Disabled
fork	p50 1501.667 us	Child creation observed; waitpid abnormal	Unsupported
posix_spawn	~4.10 ms median	Not safely tested	Built-in semantics only; not process equivalent
1 ms idle p50	CFS 129.43 us; FIFO 67.46 us	Invalid at 100 Hz	0.375 us measured, at alignment floor
1 ms idle p99	CFS 151.54 us; FIFO 89.05 us	Invalid	0.375 us measured, at alignment floor
1 ms idle max	CFS 1668.26 us; FIFO 233.75 us	Invalid	1.042 us
1 ms loaded p50	CFS 116.42 us; FIFO 64.09 us	Invalid	0.542 us
1 ms loaded p99	CFS 3741.38 us; FIFO 76.51 us	Invalid	0.583 us
1 ms loaded max	CFS 10755.59 us; FIFO 228.72 us	Invalid	0.708 us
Loaded missed periods	CFS 6.42%; FIFO 0	Invalid	0

Values are the retained runs documented during the experiment. Where the test methodology or feature state was not comparable, the table explicitly says so rather than substituting a different primitive.

Appendix B. Reproducibility and Build Artifacts

B.1 NuttX source state

```
NuttX base:      b7c8430de698e43386683e12f10680c48e90ba26
nuttx-apps:      cc3c3aa3169b9d2a137f271ccbb84e4ab590ef75
Local D1 commit: aa353a1b2a55cb0d6b8a5f51a7055373017cc907
Target:          lichee-rv-86-panel:nsh
Toolchain:       riscv-none-elf-gcc 14.3.0
Load / entry:    0x40200000 / 0x40200000
```

B.2 Key NuttX artifacts

Artifact	Purpose	SHA-256
nuttx-d1.img	First hardware-bootable local D1 image	abe8571d701f7676d4b96aa1a453f179b2a1a17490c53f920a911384c2650e9e
nuttx-d1-final.img	Stable native-timer/watchdog baseline	1f3210b7918f9091df57e4d763303e8445a378534120019d4d7c49f7e9c98a4e
nuttx-d1-benchmark.img	Initial benchmark integration	ce35ae76c97c91adbb3a4c1f7aa192778c45f2ddf61b866f924d561df669b913
nuttx-d1-benchmark-v2.img	Phase-aligned latency methodology	49adef905b4fa17492ec231fd82d10383eb60920839ef7ee6c2d15d76ac3bbef
nuttx-d1-benchmark-v3.img	Silent measurement interval	4951f186ebe23a861a451e82ce52a02ccd88f15f6b63ebfd0d61ed92163b3fd6
nuttx-d1-benchmark-v4.img	Final 23,999-count Timer1 correction	ca0d0f6cf7970da30552dfb823b1f999c352007763c822f6b4e62db2e4e6d51e

B.3 NuttX build sequence

```
cd ~/nuttx-d1/nuttx
export PATH=~/toolchains/nuttx-riscv-none-elf-14.3.0-1/bin:...
make distclean
tools/configure.sh lichee-rv-86-panel:nsh
make olddefconfig
make savedefconfig
make V=1 -j1
make clean
make -j8
```

B.4 RT-Smart source state

The RT-Smart D1 port was developed against RT-Thread v5.0.2 on branch `d1-benchmark-v5.0.2`. The D1 bring-up fix was submitted as PR **RT-Thread/rt-thread#11714** from `lanceharvie:fix/allwinner-d1-rtsmart`. The benchmark work used the RT-Thread-provided RISC-V musl toolchain under `~/toolchains/rt-thread-v1.7/...`.

B.5 Linux benchmark toolchain

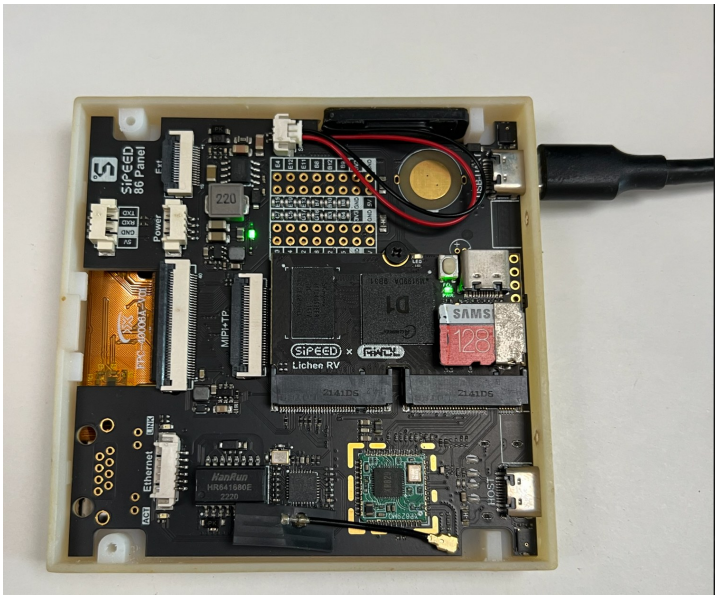
Linux benchmark binaries were built natively with GCC 13.3 using `-O2 -march=rv64gc -mabi=lp64d -static`. The CPU governor was held at the 1.008 GHz performance setting during the retained runs.

References

- [1] Allwinner D1-H User Manual, Timer and watchdog chapters. https://cs107e.github.io/readings/d1-h_user_manual_v1.0.pdf
- [2] Linux mainline device tree: arch/riscv/boot/dts/allwinner/sun20i-d1s.dtsi and sun20i-d1.dtsi.
- [3] Linux mainline D1 timer binding/SoC DTS: arch/riscv/boot/dts/allwinner/sunxi-d1s-t113.dtsi.
- [4] Linux sun4i timer driver: drivers/clocksource/timer-sun4i.c.
- [5] Linux sunxi watchdog driver: drivers/watchdog/sunxi_wdt.c.
- [6] Apache NuttX repository, tested base commit b7c8430de698e43386683e12f10680c48e90ba26. https://github.com/apache/nuttx
- [7] Apache nuttx-apps repository, tested commit cc3c3aa3169b9d2a137f271ccbb84e4ab590ef75. https://github.com/apache/nuttx-apps
- [8] RISC-V Supervisor Binary Interface specification, TIME extension and BASE extension definitions. https://github.com/riscv-non-isa/riscv-sbi-doc
- [9] OpenSBI project. The running board firmware identified itself as OpenSBI 1.3 / SBI specification 1.0. https://github.com/riscv-software-src/opensbi
- [10] RT-Thread project and D1 RT-Smart bring-up contribution PR RT-Thread/rt-thread#11714. https://github.com/RT-Thread/rt-thread

Closing statement

The most useful output of this work is not a winner. It is a repeatable engineering method for separating operating-system architecture, board-port quality, timer correctness, scheduler policy, and measurement validity. On this D1 board, those layers mattered enough to change conclusions by orders of magnitude.



Final hardware platform used throughout the experiment.

20. Post-Benchmark Linux HMI Bring-Up

After the operating-system benchmark was completed, the same 86 Panel was taken through a second phase focused on the hardware that had deliberately been outside the benchmark scope: the LCD, framebuffer, backlight, capacitive touch interface and XR829 Wi-Fi. The purpose was to prove that the board was not only a serial-console benchmark target, but a usable interactive embedded panel.

Figure 10. Final 720 x 720 panel running the Home mock dashboard in the bench setup.

Area	Final working state
Boot / OS	Tina / MaixLinux boots reliably from microSD.
LCD	720 x 720 stable output at approximately 58.1 fps; correct colours and no flicker.
Panel profile	`default_lcd` is correct for this unit; `nv3052c_rgb` is not.
Framebuffer	Black, white, red, green and blue test fills verified correctly.
Backlight	PWM7 at 20 kHz.
Touch	`fts_ts` generates live X/Y input events and supports direct on-screen navigation.
Wi-Fi	XR829 connected at 192.168.0.13; routing, DNS and Internet connectivity verified.
Credential hygiene	Wi-Fi password scrubbed from local logs before the work was preserved.

Result
The display, touch and network path is now a working application platform rather than an unverified peripheral set. These changes do not alter the benchmark numbers in Sections 4-18; they extend the hardware bring-up beyond the original benchmark scope.

20.1 Display, Touch and RunTime System

The display work also resolved a panel-identity issue. On the physical unit used here, the working configuration is `default_lcd`. Once selected, the panel produced a stable 720 x 720 image with correct colour order and no visible flicker. Backlight control was established through PWM7 at 20 kHz. The framebuffer was then exercised with primary-colour and black/white fills before the higher-level UI work began.

Touch input is delivered by `fts_ts`. Coordinates were observed live and mapped directly to screen regions. A test touch at approximately X=96, Y=657 correctly landed in the bottom-left navigation area, confirming that the coordinate system aligned with the displayed geometry.

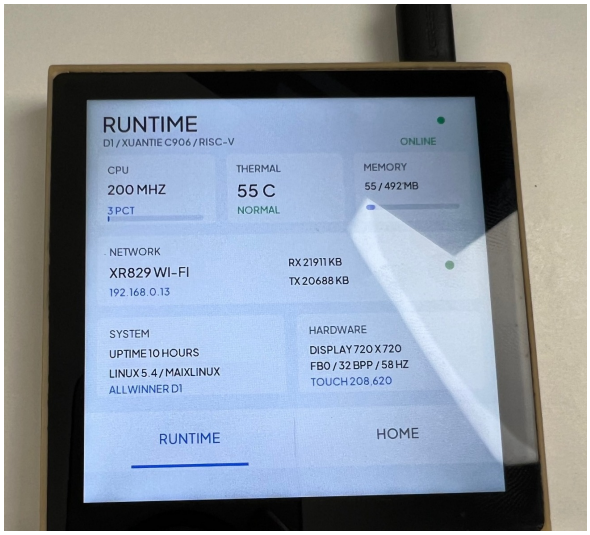


Figure 11. Light-theme RunTime status screen using live Linux system and network information.

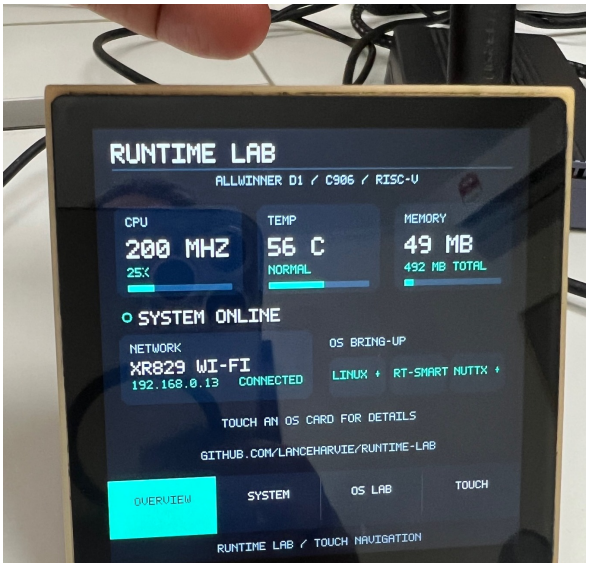


Figure 12. Earlier RunTime Lab dashboard prototype with touch-oriented navigation.

The RunTime screen demonstrates that the panel can expose useful live system state rather than simply draw static test graphics. The visible application data includes CPU frequency/load, thermal state, memory usage, XR829 Wi-Fi status, IP address, network traffic, uptime, Linux/MaixLinux identification, framebuffer information and the most recent touch coordinate.

- **Display:** native 720 x 720 layout designed around the square panel rather than a desktop UI scaled down to fit.
- **Input:** touch regions are large enough for direct finger use, with bottom navigation deliberately occupying a wide band.
- **RunTime data:** stem values are read from the Linux environment and refreshed without requiring a browser or

- **Performance:** the screen is mostly static, so the application only needs to redraw state that changes rather than maintain a high-frequency animation loop.

20.2 RunTime / Home Touch Interface

The UI was then extended from a diagnostic dashboard into a two-screen touch application. The bottom navigation switches between **RunTime** and **Home** without rebooting the board or restarting the graphical application. The Home screen is intentionally a mock smart-home dashboard. It is not connected to a Home Assistant server at this stage.

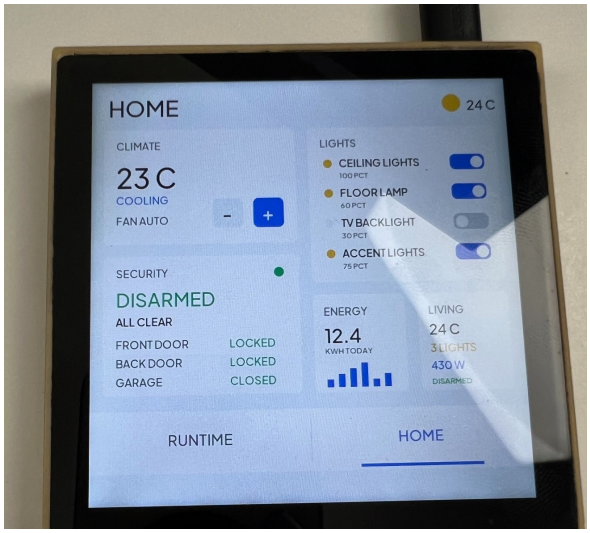


Figure 13. Home mock dashboard. Climate, lighting, security and energy controls are local UI state.

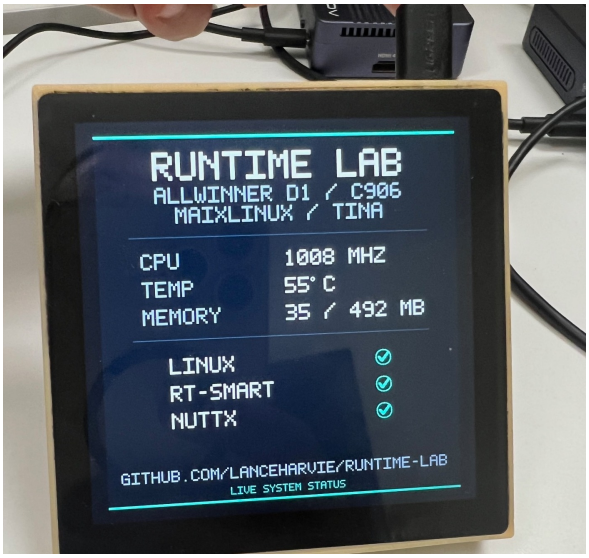


Figure 14. Earlier OS-lab presentation screen showing the three operating systems used in the benchmark.

Home dashboard behavior

- **Climate:** target temperature can be adjusted locally with large + / - touch controls; cooling state and fan mode are represented visually.
- **Lights:** ceiling, floor, TV backlight and accent-light switches can be toggled locally as mock state.
- **Security:** front door, back door and garage state are presented in a compact at-a-glance card.
- **Energy and living-room summaries:** provide a small graphical consumption view and consolidated environmental/status values.
- **Navigation:** the RunTime and Home tabs occupy the bottom of the screen and make the panel usable as both a system demonstrator and an application mockup.

> **Current boundary**
> The Home screen deliberately stops at the UI/state layer. A future revision can map the same controls to real Home Assistant REST or WebSocket entities without redesigning the display, but no Home Assistant service or browser runtime is required for the present demonstrator.

Appendix C. RunTime Panel Source and Reproducibility

The follow-up panel application has been preserved as a separate public source artifact so the HMI work can be reproduced independently of the benchmark images and operating-system ports.

Repository	github.com/lanceharvie/lichee-rv86-runtime-panel
Visibility	Public
Branch	main
Retained commit	`20c2a238cada864726ce26bc5c0829dd10b65afb`
Local worktree	Clean at the retained state
Included	Source, Makefile, README, font generator / atlas, MIT license and font license
Excluded	Credentials and the local build environment

Reproducibility note

The repository intentionally excludes credentials and machine-specific build state. This keeps the public artifact focused on the application and its reproducible source inputs while avoiding accidental disclosure of Wi-Fi secrets or local environment details. The exact retained commit above should be used when correlating screenshots or future changes with this report update.

> **Updated closing statement**
> The benchmark established how Linux, RT-Smart and NuttX behave on the same D1/C906 hardware. The follow-up work proves a second point: the same board can also be taken through a practical HMI bring-up, from raw framebuffer and touch validation to a working networked 720 x 720 touch application. The benchmark and panel work should be treated as complementary results rather than a single performance ranking.